

Variables & Primitive Data Types

  RamRodz Robotics · Team 6366 · Programming Worksheet



Name

Date

  RAM RODZ ROBOTICS | TEAM #6366 | HILLCREST & WOODMONT HIGH SCHOOLS ·
SIMPSONVILLE, SC

 **Focus Check-In — How are you feeling right now?**



Struggling



Okay



Ready



TODAY'S BIG QUESTION

The robot needs to remember how fast its motors are spinning, whether a button is being held, and how many shots it has fired — all at the same time. How does the code keep track of all of that?

P Parking Lot — Random Thoughts

If your brain jumps to a question or idea, write it here so you don't lose it — ask later!

1 Key Words — Check Off When Explained

Variable

A named container in memory that holds a value. You can read it, change it, and use it anywhere in your code.

Data Type

Tells Java *what kind* of value a variable can hold — a whole number, a decimal, true/false, etc.

int

A whole number with no decimal point.
Examples: 0, 5, -3, 1000. Used for counts, port numbers, and indexes.

double

A number that *can* have a decimal point.
Examples: 0.0, 0.75, -1.5. The go-to type for motor speeds and sensor values.

boolean

Can only be true or false — nothing else.
Used for button states, flags, and on/off conditions.

Declare / Assign

Declare = create a variable and give it a type.

Assign = put a value into it using =.

2 What is a Variable?



Listen — A variable is like a labeled box in memory. You give it a name, put a value in it, and you can change that value any time.

ANATOMY OF A VARIABLE DECLARATION

```
int  driveSpeed  =  5;
↑      ↑        ↑   ↑
type  name  assign value
```

Plain English: **type** = what kind of value. **name** = what you call it. **=** = "put this in the box." **value** = what goes in. The semicolon ; ends the line.

Naming rules: Start with a lowercase letter · No spaces (use camelCase: driveSpeed) · No special characters except _ · Make it *descriptive* — shooterSpeed is better than x.

 **Quick Question — Label each part of this line:**

3 int — Whole Numbers

- ☐ Listen — Use int whenever the value is always a whole number: port numbers, button indexes, shot counters, loop counts.

Java — int in FRC

```
int motorChannel = 0;    // left drive motor is on port 0
int buttonIndex  = 1;    // the A button is at index 1
int shotsFired   = 0;    // start the counter at zero
int autoStepCount = 3;    // autonomous has 3 steps
```

 Plain English: **ints can't hold decimals.** If you try to store 0.5 in an int, Java cuts off the decimal and keeps only 0. Use double (next section) for anything that needs a fraction.

 **Try It — Write an int variable that stores the number of balls loaded in the robot (start at 0)**

4 double — Decimal Numbers

- ☐ Listen — double is the most common type in FRC robot code. Motor speeds, sensor distances, and joystick values all need decimals.

Java — double in FRC

```
double shooterSpeed = 0.8; // 80% power (motors go 0.0 to 1.0)
double driveThrottle = 0.0; // start stopped
double distanceInches = 24.5; // sensor reading: 24.5 inches away
double turnRate       = -0.3; // negative = turning left
```

💡 Plain English: **Motor speeds in WPILib are always doubles between -1.0 and 1.0.** Full forward = 1.0, full reverse = -1.0, stopped = 0.0. That's why double shows up everywhere in drive code.

✎ Try It — Write a double variable for a shooter speed set to 75% power

5 boolean — True or False

☐ Listen — A boolean can only ever be true or false. It's perfect for anything that is either on or off, pressed or not pressed, done or not done.

Java — boolean in FRC

```
boolean buttonPressed = false; // A button is not being held down
boolean isShooting    = false; // shooter is currently off
boolean autoFinished  = false; // autonomous routine not done yet
boolean limitHit      = true;  // limit switch has been triggered
```

💡 Plain English: **true and false are written in all lowercase** — Java is case-sensitive. True or TRUE won't work. Boolean variables are often named like a yes/no question: `isRunning`, `buttonPressed`, `hasTarget`.

✎ Try It — Write a boolean that tracks whether the intake motor is running (start as not running)

6 String — Text Values

☐ Listen — A String holds text: words, sentences, labels, messages. Unlike `int` or `boolean`, it is *not* a primitive — notice it starts with a capital S.

Java — String in FRC

```
String robotStatus = "Ready";           // current status to show on dashboard
String autoRoutine = "Drive Forward";   // which autonomous path to run
String teamName    = "RamRodz";         // team name — won't change
String shooterState = "Off";            // shooter status message
```

💡 Plain English: **Text always goes inside double quotes " "**. Forget the quotes and Java thinks Ready is a variable name, not the word "Ready" — and your code won't compile.

Always: `String label = "some text";`

Combining Strings with +

You can glue a String and another variable together using `+`. This is called **concatenation**.

```
String message = "Speed: " + shooterSpeed;
```

If `shooterSpeed` is `0.8`, message becomes "Speed: 0.8"

```
String label = "Team " + teamNumber;
```

If `teamNumber` is `6366`, label becomes "Team 6366"

Java — Reassigning a String

```
String shooterState = "Off";           // starts as "Off"
shooterState = "Spinning Up";         // update the message (no "String" again)
shooterState = "Ready to Fire";       // update again when fully spun up
```

💡 Plain English: **Strings update exactly like any other variable** — write the name and the new value, no type keyword again. The old text is replaced by the new text.

✏️ **Try It — Write a String variable for the robot's current game mode. Start it as "Disabled".**

 **Concatenation** — Write a line that builds a status message combining the text "Balls loaded: " with an int variable named ballCount

7 Changing a Variable's Value

- ☐ Listen — After a variable is declared, you can change its value any time. Leave out the type — just write the name and the new value.

Java — Reassigning Variables

```
double motorSpeed = 0.0;    // declare and set to 0 (stopped)
motorSpeed = 0.75;          // now running at 75% — no "double" needed
motorSpeed = 0.0;           // stopped again

boolean buttonDown = false; // button not pressed
buttonDown = true;          // button is now held — no "boolean" needed
```

 Plain English: **The type is written once — when you first create the variable.** After that, just write name = newValue; to update it. Writing the type again is an error: Java would think you're trying to create a second variable with the same name.

Quick Reference — Java Types

Type	Holds	Example values	FRC use case
int	Whole numbers	0, 5, -3, 1000	Port numbers, counters, button indexes
double	Decimal numbers	0.0, 0.75, -1.0	Motor speeds, sensor distances, throttle
boolean	True or false only	true, false	Button states, flags, limit switches
String	Text (not a primitive)	"Ready", "Auto"	Dashboard labels, mode names
long	Very large whole numbers	1000000L	Timestamps, encoder ticks over time

 **Quick Question — Which type would you use for each? Write int, double, boolean, or String.**

Robot Scenario — Set Up the Variables

The Challenge

The programming team needs to set up the starting variables for a robot's intake system. The intake needs to track:

- The **motor speed** for the intake roller (starts stopped)
- Whether the **intake button** is currently being held (starts not pressed)
- The **port number** of the intake motor (it's on port 2)
- The **number of game pieces** collected so far (starts at zero)
- A **status message** String for the dashboard (starts as "Idle")



I can identify the right type for each: speed → double, button → boolean, port → int, count → int, status message → String.



Write It — Declare all five variables with their correct types and starting values



Bonus — Write the reassignment lines to update the speed to 0.6, mark the button as pressed, and update the status message to "Running"

✓ End of Lesson — I Can...

- ☐ Explain what a variable is and why code needs them — the robot has to **remember** values like speed and button state.
- ☐ Write a variable declaration with the correct syntax: **type name = value;**
- ☐ Choose the right type for a given situation: **int** for whole numbers, **double** for decimals, **boolean** for true/false.
- ☐ Update a variable's value after it's been declared — without writing the type again.
- ☐ Declare and use a **String** variable — wrap text in double quotes, use + to combine it with other variables.
- ☐ Name variables clearly using **camelCase** so other programmers (and future me) know what each one holds.